

综合实验：计算机科学基础概念可视化探索

实验背景：

随着大语言模型的爆发，Vibe Coding（氛围编程）正在重塑现代软件开发范式。Vibe Coding 强调通过自然语言精准表达设计意图，借助 AI 快速构建代码实现，从而让开发者将核心精力从“纠结语法细节”释放到“深化概念理解、架构逻辑思考与高效团队协作”上。本实验旨在让计算机专业新生体验现代软件工程的标准流程，以 Vibe Coding 的高效编程范式，结合版本控制与团队协作工具，以小组为单位完成一个轻量级计算机基础概念可视化程序的开发与开源交付。

实验目标：

1. 掌握 Markdown 语法，编写结构清晰的工程文档。
2. 掌握 Git 分布式版本控制的核心操作及基于代码托管平台（如 Gitee/GitHub）的团队协作流程。
3. 掌握大语言模型（LLM）的提示词技巧，利用 AI 辅助代码生成、架构构建与调试。

实验环境与工具准备

- **版本控制与代码托管：**Git，以及任意代码托管平台（如 Gitee、GitHub、GitLab 等）。
- **开发环境 / 编辑器：**自由选择，如 Visual Studio Code、Cursor、PyCharm、JetBrains 系列或 Sublime Text 等。
- **AI 编程助手：**可自由组合使用 Web 端大模型（如 DeepSeek、Kimi、Qwen 等）或集成于 IDE 中的 AI 辅助插件/AI 原生编辑器（如 Cursor、Trae / GitHub Copilot、Continue、Roo Code 等）。
- **开发语言与界面库：**推荐使用 Python（可选用 Tkinter、Pygame 等库，降低 GUI 开发门槛），也允许使用团队熟悉的其它编程语言与可视化框架。

一、团队组建与需求文档（Markdown 与 Git 基础）

任务描述：

学生需组建 4-6 人的开发团队，完成项目初始化，并协同编写需求文档。

具体步骤：

1. **仓库初始化**：团队负责人在 Gitee/GitHub 上创建项目仓库，初始化 README.md 和 .gitignore 文件，并将团队成员添加为开发者。
2. **克隆与分支**：所有成员将仓库 clone 到本地，并各自创建并切换到自己的功能分支（如 docs/feature-proposal）。
3. **Markdown 文档协作**：
 - 成员分工使用 Markdown 编写项目需求与设计文档 **Project_Requirements_and_Design.md**。
 - **排版要求**：必须包含多级标题、项目列表、加粗强调、至少一张参考图片，以及用于分工安排的 Markdown 表格。
 - **选题建议**：演示程序需围绕《课程讲义》中的基础知识。例如：**进位制转换步骤演示器**（清晰展示不同进位制之间转换的详细步骤）、**补码加减法运算模拟器**（重点展示求补过程和溢出判断）、或**ASCII码和奇偶校验的发送和接收**。也可选择人工智能与机器学习的基础算法，例如：**线性回归与梯度下降步骤演示器**（动态展示损失函数收敛与参数更新轨迹）、**K-Means 聚类算法迭代过程模拟**（展示质心更新与簇分配），亦可选择其他计算机基础知识（如经典排序/查找算法、进程调度、页面置换等）的可视化。
4. **提交与合并**：成员各自 commit 文档的修改，推送到远程仓库，并发起 Pull Request (PR)。团队负责人进行代码审查 (Code Review) 并合并到主分支。

二、 选题审核

任务描述：

各小组在完成初步的需求文档后，需提交进行严格的结构与方向把关，确保项目的技术深度足够且工作量饱满。

具体步骤：

1. 文档提交

团队负责人将合并后的 **Project_Requirements_and_Design.md** 提交给教师进行审核。

2. 教师评审

教师将从学术规范性、逻辑表达清晰度、选题的实际工作量，以及可视化方案的可行性等维度进行评审，并给出“通过”、“需修改”或“驳回重做”的明确意见。

3. 意见响应

未通过的小组需根据教师给出的修改意见，在指定时间内更新文档，并再次提交审核，直至“通过”后方可进入代码开发阶段。

三、 AI 驱动的代码构建（Vibe Coding 实践）

任务描述：

学生需运用 Vibe Coding 理念，通过高质量的提示词（Prompt）和对话引导 AI 编程助手完成可视化演示程序的核心代码开发。此阶段重点考核意图表达、提示词工程能力与代码审查/Debug 能力。

具体步骤：

1. 提示词工程与意图表达（Prompt Engineering）：

- 撰写结构化的系统提示词，让 AI 生成符合要求的界面与逻辑代码。
- 提示词需包含：角色设定（如“资深 GUI 开发者”）、具体需求（如“使用 Python 绘制一个 800x600 的窗口，展示逻辑移位与算术移位的区别”）、约束条件（如“代码需包含详细的中文注释，代码结构要模块化”）。

2. 代码生成、验证与 Vibe Debug：

- 将 AI 生成的代码引入本地工程进行运行验证。
- 遇到报错或效果不符合预期时，将错误日志（Error Traceback）反馈给 AI，通过多轮自然语言对话引导 AI 进行错误排查与代码优化。

3. “AI 结对编程”记录：

- 在项目中新建一个 **AI_Prompts_Log.md** 文件，记录成员向 AI 提问的最具代表性的 3 次提示词、AI 的原始输出，以及人工介入微调/修复的部分。

四、 代码集成与冲突解决（Git 高级协作）

任务描述：

各成员将自己负责的模块（如：UI 界面模块、核心计算模块、事件响应模块）集成到主程序中，并真实体验冲突解决。

具体步骤：

1. 并行开发与提交：

- 成员在各自的 `feature/*` 分支上完成 AI 辅助生成的代码，并推送到远程。

2. 制造与解决冲突：

- 建议人为制造一次合并冲突（例如两名成员同时修改了主窗口的初始化参数）。
- 需在本地通过 `git pull` 或 `git fetch + git rebase` 获取最新代码，触发冲突。
- 在 VS Code 中手动识别 <<<<<< HEAD 冲突标记，协商保留正确的代码，完成合并并最终推送到远程 `main` 分支。

五、项目交付与开源规范

任务描述：

完善最终的项目主页，培养开源社区规范意识。

具体步骤：

1. 完善 README.md：

- 添加项目的安装与运行步骤。
- 添加软件运行的高清截图（复习 Markdown 图片语法）。
- 添加**贡献者名单**及开源协议声明。

2. 成果演示：各小组在课堂上拉取最新 `main` 分支代码，运行桌面演示程序，并简述团队的 Git 工作流和使用 Vibe Coding 开发的心得体验。

六、实验考核与评分标准（100分）

考核维度	评分细则	占比
文档规范 (Markdown)	README.md 及设计文档结构清晰，正确使用了标题、列表、表格、图片、代码块等语法。	20%
协作流程 (Git)	提交记录（Commit log）清晰规范； 正确使用了分支隔离开发； 成功处理了合并冲突并有相关记录。	30%
Vibe Coding 与 AI 应用	AI_Prompts_Log.md 记录详实，提示词设计合理；能借助 AI 快速解决开发痛点，同时能识别并修正 AI 的“幻觉”或低级错误，非盲目复制粘贴。	25%

考核维度	评分细则	占比
程序实现与展示	演示程序可成功运行，无明显 Bug；界面交互友好，能清晰直观地展示选定的计算机基础概念。	25%

七、 最终提交文件列表

为确保项目质量可被准确评估，各小组需在实验截止前提交以下交付物：

- **代码托管仓库链接**（需包含完整的 commit 历史、分支记录与 Pull Request 记录，若设置为私有请提前给教师赋予访问权限）。
- **Project_Requirements_and_Design.md**（Markdown 格式，需体现选题审核阶段的修改完善情况）。
- **AI_Prompts_Log.md**（AI 结对编程与 Vibe Coding 过程记录）。
- **完整的程序源代码**（包含界面构建逻辑与底层计算逻辑的文件，如 .py 源文件）。
- **README.md**（包含图文并茂的运行指南与**团队分工信息**）。